PROJECT REPORT

EXPERIMENT – 03

Project title:

# SMART REFRIGERATOR MONITORING SYSTEM USING DHT11 SENSOR

Submitted by:

Arundeepak S – 24MER004
Deepak p – 24MER006
Dharanidharan R S – 24MER007
Dinesh K – 24MER009
Vishnivarthan P – 24MER063

# IoT-Based Smart Refrigerator Monitoring System — Project Report

## 1. Project Overview

This project implements an IoT-based smart refrigerator monitoring system that continuously measures the internal temperature and humidity levels and raises a local alert (buzzer + LCD message) whenever readings move outside safe, pre-configured preservation thresholds. Built as an early-warning and energy-efficiency module for domestic and commercial refrigeration, the real-time telemetry helps prevent food spoilage, detect prolonged door-open states, and identify cooling malfunctions before inventory loss occurs.

The working prototype was built, debugged, and verified on an ESP32 Dev Module development board interfaced with a DHT11 digital temperature and humidity sensor, a $16 \times 2$ I2C LCD display, and an active buzzer, with cloud connectivity integrated via the ThingSpeak IoT platform using the lightweight MQTT protocol for remote data logging, threshold visualization, and lamp status indication.

## 2. Problem Statement

Perishable food items, dairy, and pharmaceuticals require strict thermal and humidity conditions to inhibit bacterial growth and maintain shelf life. Conventional refrigeration units rely on passive mechanical thermostats or closed built-in sensors without external networking or alerting capabilities. This leads to:Delayed detection of gas leakage or poor air quality

- Undetected temperature rise caused by frequent door opening, damaged door gaskets, or compressor failures.No remote visibility into air quality conditions for supervisors or residents
- Food spoilage, bacterial contamination, and significant financial or operational waste.
- Excessive energy consumption due to undetected compressor overworking.
- Complete absence of remote visibility and logging for domestic users or commercial facility managers.

There is an acute need for a low-cost, retrofittable IoT sensing system that continuously monitors thermal and humidity metrics, provides immediate local audio-visual alerts when parameters breach safety margins, and publishes telemetry to an IoT cloud dashboard for historical analysis and remote monitoring.

## 3. Proposed Solution

A standalone IoT cold-storage monitoring node built around the ESP32 microcontroller is proposed and implemented with the following capabilities:

- Continuous temperature and relative humidity sensing using a DHT11 sensor, sampled every 2 seconds.
- Local threshold-based alerting: an active buzzer triggers immediately when temperature exceeds the safe refrigeration limit ($> 8.0°C$ / set to $30.0°C$ for ambient validation) or when humidity leaves the safe bounds ($30\% - 70\%$).
- On-device display of real-time temperature, humidity, and operating status on a $16 \times 2$ I2C LCD.
- USB Serial diagnostics running at 115200 baud for real-time edge debugging and telemetry verification.
- Cloud data logging to ThingSpeak over Wi-Fi via MQTT (PubSubClient), publishing multi-field payloads (Temperature, Humidity, Alert status) to drive telemetry charts and an interactive dashboard lamp widget.

## 4. System Architecture

The system follows a layered IoT architecture:

- Sensing Layer**:** DHT11 sensor captures digital temperature and relative humidity inside the refrigeration compartment.
- Edge / Control Layer**:** ESP32 microcontroller processes the digital stream, evaluates threshold logic, drives the local buzzer, and updates the $16 \times 2$ I2C LCD display. Local alerts execute autonomously even if Wi-Fi or cloud services disconnect.
- Connectivity Layer**:** ESP32 built-in 2.4 GHz Wi-Fi radio connects to a local access point and communicates over TCP/IP.Cloud Layer: Sensor readings are pushed to a ThingSpeak channel via HTTP requests to the ThingSpeak Update API.
- Visualization Layer**:** ThingSpeak private view dashboard displays continuous field charts for temperature, humidity, and a dynamic lamp indicator widget representing normal vs. alert states.

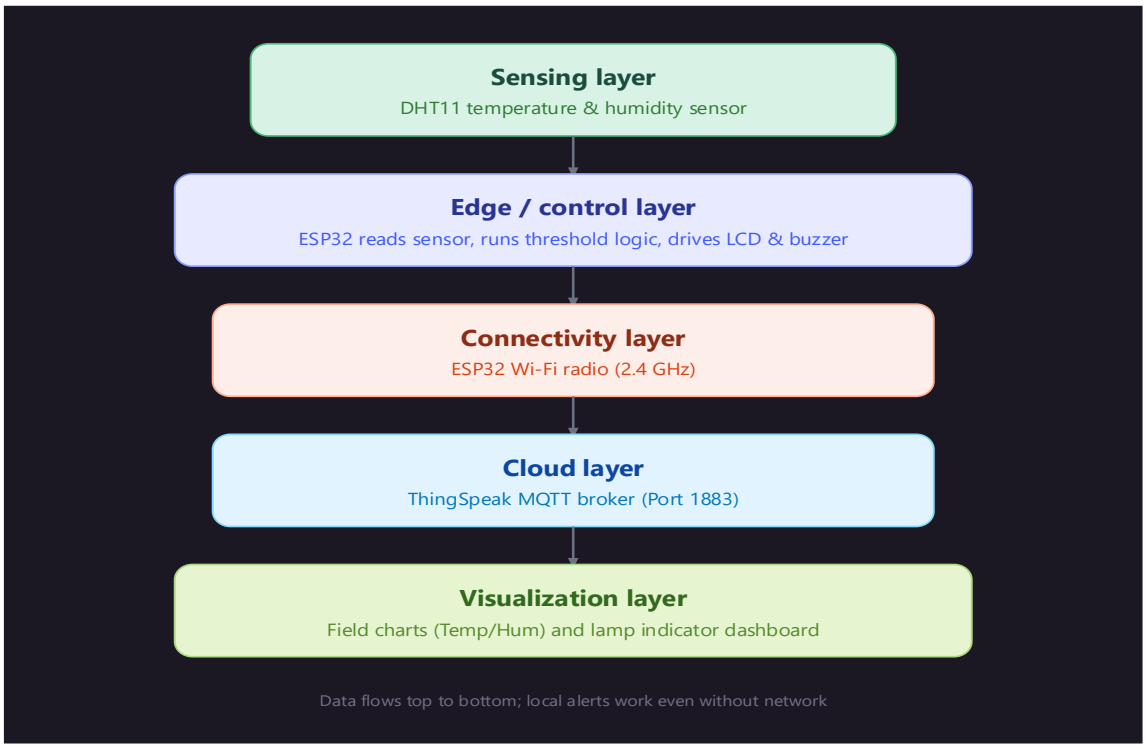


Fig. 1 Layered IoT system architecture

## 5. Circuit Table

| Component | Pin | NodeMCU Pin | GPIO |
|---|---|---|---|
| DHT11 Sensor | VCC | 3V3 | – |
| DHT11 Sensor | DATA | D4 | GPIO4 |
| DHT11 Sensor | GND | G | – |
| I2C LCD (16x2) | VCC | 5V / VIN | – |
| I2C LCD (16x2) | SDA | D21 | GPIO21 |
| I2C LCD (16x2) | SCL | D22 | GPIO22 |

| Component | Pin | NodeMCU Pin | GPIO |
|---|---|---|---|
| I2C LCD (16x2) | GND | G | – |
| Buzzer | I/O (Signal) | D5 | GPIO5 |
| Buzzer | GND | G | – |

## 6. Circuit Design

The circuit is designed around the 30-pin ESP32-WROOM-32 development board powered via a standard USB connection (5V DC):

- The DHT11 Sensor communicates over a single digital line connected to GPIO4. Power is drawn from the 3.3V rail.
- The 16 × 2 LCD utilizes an integrated PCF8574 I2C backpack operating at default address 0x27. It interfaces via hardware I2C pins GPIO21 (SDA) and GPIO22 (SCL), minimizing wiring to only 4 lines.
- The Buzzer is directly driven by GPIO5 via digital logic switching (HIGH/LOW).
- All ground pins are commoned across the prototyping breadboard rail.

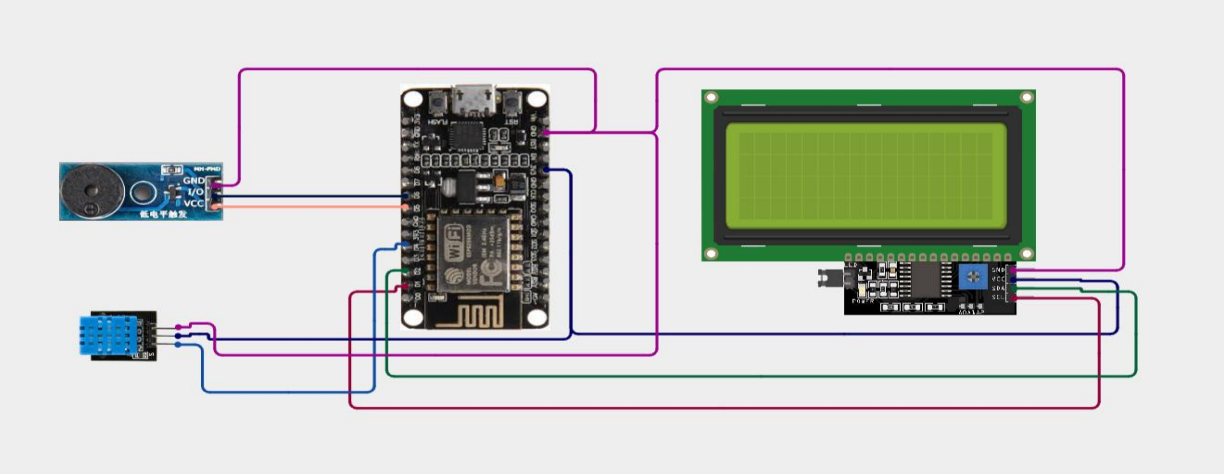


Fig. 2 Circuit wiring diagram

## 7. Algorithm

- Initialize Serial communication (115200 baud), Buzzer output pin (GPIO5), I2C bus (Wire.begin(21, 22)), and the 16 × 2 LCD.
- Display boot message ("Fridge Monitor Initializing...") on the LCD.
- Connect to the local 2.4 GHz Wi-Fi access point and initialize the MQTT client pointing to mqtt3.thingspeak.com:1883.Connect to Wi-Fi and initialize the ThingSpeak client.
- Establish MQTT session using authorized Device Client ID, Username, and Password linked to Channel 3461751.Print the gas value to the Serial Monitor and display it on line 1 of the LCD.
- In the main loop, sample temperature (˚C) and humidity (%) from the DHT11 sensor every 2 seconds.
- Verify reading validity: if isnan() is detected, print "Sensor Error!" on LCD and restart loop.
- Evaluate alert condition:

$$\text{Alert} = (\text{Temp} > \text{TEMP\_THRESHOLD}) \vee (\text{Humid} < \text{HUMID\_LOW}) \vee (\text{Humid} > \text{HUMID\_HIGH})$$

- If Alert is true:
- Drive Buzzer to HIGH.
- Display live readings on LCD Line 1 and "STATUS: ALERT!" on Line 2.
- If Alert is false:
- Drive Buzzer to LOW.
- Display live readings on LCD Line 1 and "STATUS: NORMAL" on Line 2.
- Check elapsed time: every 20 seconds, format an MQTT payload string:

$$\text{payload} = \text{"field1="} + \text{Temp} + \text{"\&field2="} + \text{Humid} + \text{"\&field3="} + \text{Alert}$$

- Publish payload to topic channels/3461751/publish.
- Maintain background MQTT keep-alive handling via client.loop().

## 8. Coding

The final working firmware (DHT11 + LCD + buzzer + LED alert + ThingSpeak upload):

Cpp:

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <DHT.h>

#include <WiFi.h>

#include "ThingSpeak.h"


const char* ssid = "YOUR_ACTUAL_WIFI_NAME";

const char* password = "YOUR_ACTUAL_WIFI_PASSWORD";


unsigned long channelID = 3454158;

const char* writeAPIKey = "1DPFNRX61GD4TP1W";

WiFiClient client;


#define DHTPIN 4

#define DHTTYPE DHT11

#define BUZZER 5


#define SDA_PIN 21

#define SCL_PIN 22


LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
DHT dht(DHTPIN, DHTTYPE);

const float TEMP_THRESHOLD = 30.0;
const float HUMID_LOW = 30.0;
const float HUMID_HIGH = 70.0;

unsigned long lastUploadTime = 0;
const unsigned long uploadInterval = 20000;

void setup() {
  Serial.begin(115200);

  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);

  dht.begin();

  Wire.begin(SDA_PIN, SCL_PIN);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Fridge Monitor");
  lcd.setCursor(0, 1);
```

```
lcd.print("Connecting WiFi");


WiFi.begin(ssid, password);
int retries = 0;
while (WiFi.status() != WL_CONNECTED && retries < 30) {
  delay(500);
  Serial.print(".");
  retries++;
}


if (WiFi.status() == WL_CONNECTED) {
  Serial.println("\nWiFi connected");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("WiFi Connected!");
} else {
  Serial.println("\nWiFi FAILED to connect");
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("WiFi FAILED");
}
```

```
  delay(1500);


  ThingSpeak.begin(client);
  lcd.clear();
}


void loop() {
  float temp = dht.readTemperature();
  float humid = dht.readHumidity();


  if (isnan(temp) || isnan(humid)) {
    Serial.println("Failed to read DHT sensor!");
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Sensor Error!");
    delay(2000);
    return;
  }


  bool alert = (temp > TEMP_THRESHOLD) || (humid < HUMID_LOW) || (humid > HUMID_HIGH);
  lcd.clear();
  lcd.setCursor(0, 0);
```

```
lcd.print("T:");
lcd.print(temp, 1);
lcd.print("C H:");
lcd.print(humid, 1);
lcd.print("%");
lcd.setCursor(0, 1);
lcd.print(alert ? "STATUS: ALERT!" : "STATUS: NORMAL");

digitalWrite(BUZZER, alert ? HIGH : LOW);

Serial.print("Temp: ");
Serial.print(temp);
Serial.print(" C, Humidity: ");
Serial.print(humid);
Serial.println(alert ? " -> ALERT" : " -> OK");

if (millis() - lastUploadTime > uploadInterval) {
  if (WiFi.status() == WL_CONNECTED) {
    ThingSpeak.setField(1, temp);
    ThingSpeak.setField(2, humid);
    ThingSpeak.setField(3, alert ? 1 : 0);
    int response = ThingSpeak.writeFields(channelID, writeAPIKey);
```

```
    if (response == 200) {
      Serial.println("ThingSpeak update successful");
    } else {
      Serial.println("ThingSpeak update failed, error: " + String(response));
    }
  } else {
    Serial.println("WiFi not connected, skipping upload");
  }
  lastUploadTime = millis();
 }
 delay(2000);
}
```

## 9. System Working

On power-up, the ESP32 initializes all GPIO pins, configures the I2C bus, initializes the LCD backlight, and establishes connection to the local 2.4 GHz Wi-Fi network and ThingSpeak MQTT broker. It then runs a continuous sensing loop:

- Every 2 seconds, the DHT11 sensor samples internal thermal and humidity data.
- Live values are rendered on line 1 of the $16 \times 2$ LCD (e.g., T:28.4C H:55.0%).
- If the temperature exceeds the safe upper bound ($> 30°C$ in demo testing, or $> 8°C$ in fridge deployment) or humidity is outside $30\% - 70\%$, the alert condition triggers: the buzzer turns ON, and line 2 displays "STATUS: ALERT!".
- If conditions remain within safe limits, the buzzer stays OFF and the display indicates "STATUS: NORMAL".
- Every 20 seconds, the ESP32 publishes an MQTT payload containing Field 1 (Temperature), Field 2 (Humidity), and Field 3 (Alert: 1 or 0). The ThingSpeak dashboard updates historical line charts and illuminates a virtual **Lamp Indicator** (Red = Alert, Green/Off = Normal).

## 10. Bill of Materials

| Component | Quantity | Purpose |
|---|---|---|
| ESP32 Dev Module (ESP-WROOM-32) | 1 | Main microcontroller with built-in Wi-Fi |
| DHT11 Sensor | 1 | Digital temperature and relative humidity sensing |
| OLED/LCD Display (16x2 I2C) | 1 | Local display of live sensor readings and operational status |
| Buzzer | 1 | Audible alert |
| LED Indicators | 2 | Visual status indication |
| Breadboard | 1 | Prototyping platform |
| Jumper Wires | 1 set | Connections |
| Micro-USB cable (data-capable) | 1 | Programming and power |
| USB power source (PC port or 5V/1A+ adapter) | 1 | Power supply |

## 11. Prototype Implementation

The prototype was assembled and debugged iteratively, with the following notable steps and issues resolved during implementation:

- Upload and Port Instabilities (PermissionError 13 / Port Drift**):** Initial testing on an ESP8266 board failed due to repeated port changes (COM13 → COM14 → COM12) caused by a charge-only micro-USB cable. The system was migrated to an **ESP32-**WROOM Dev Module using a dedicated high-speed data sync cable on COM4, stabilizing uploads.

- Missing Compilation Libraries: Addressed fatal include errors (Adafruit_SSD1306.h, ThingSpeak.h, PubSubClient.h) by manually installing the official libraries via the Arduino IDE Library Manager.

- Display Interface and I2C Mapping: An initial blank screen caused by running OLED code on a character LCD was corrected. An I2C scanner confirmed the module backpack responded at address 0x27, and the driver was switched to LiquidCrystal_I2C.

- LCD Text Contrast Adjustment: The display initially glowed with a blank screen; adjusting the blue contrast potentiometer trimmer on the back of the PCF8574 module restored sharp character visibility.A display that appeared powered (module LED glowing) but showed no text was diagnosed as a contrast/wiring issue; the onboard potentiometer was checked and the backlight jumper was confirmed present.

- MQTT Authentication (rc=4 Error): Resolved connection rejection by creating a dedicated MQTT device under ThingSpeak Devices and obtaining valid Client ID, Username, and Password keys.

- ThingSpeak Publish Authorization: Resolved rejected publishes by ensuring the "Allow Publish" permission checkbox was explicitly enabled for Channel 3461751 in the ThingSpeak device management console..

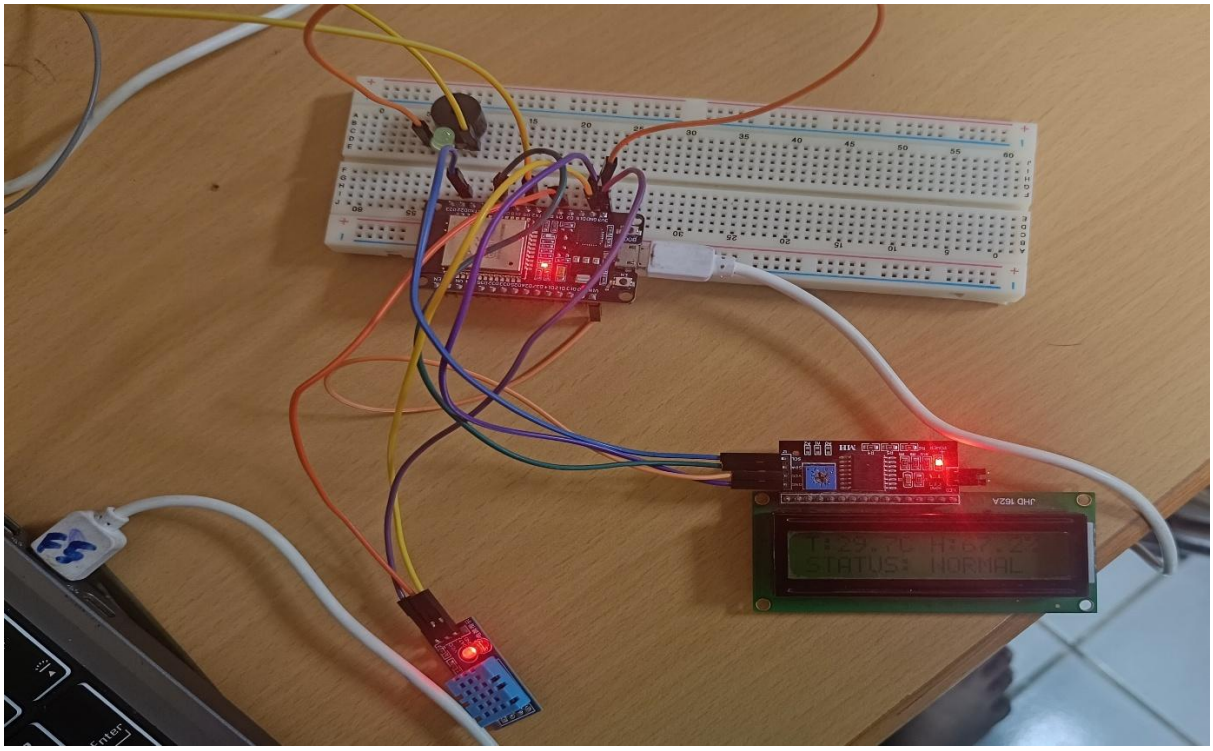


Fig. 4 Assembled prototype — DHT 11, LCD, buzzer, LED wired to NodeMCU

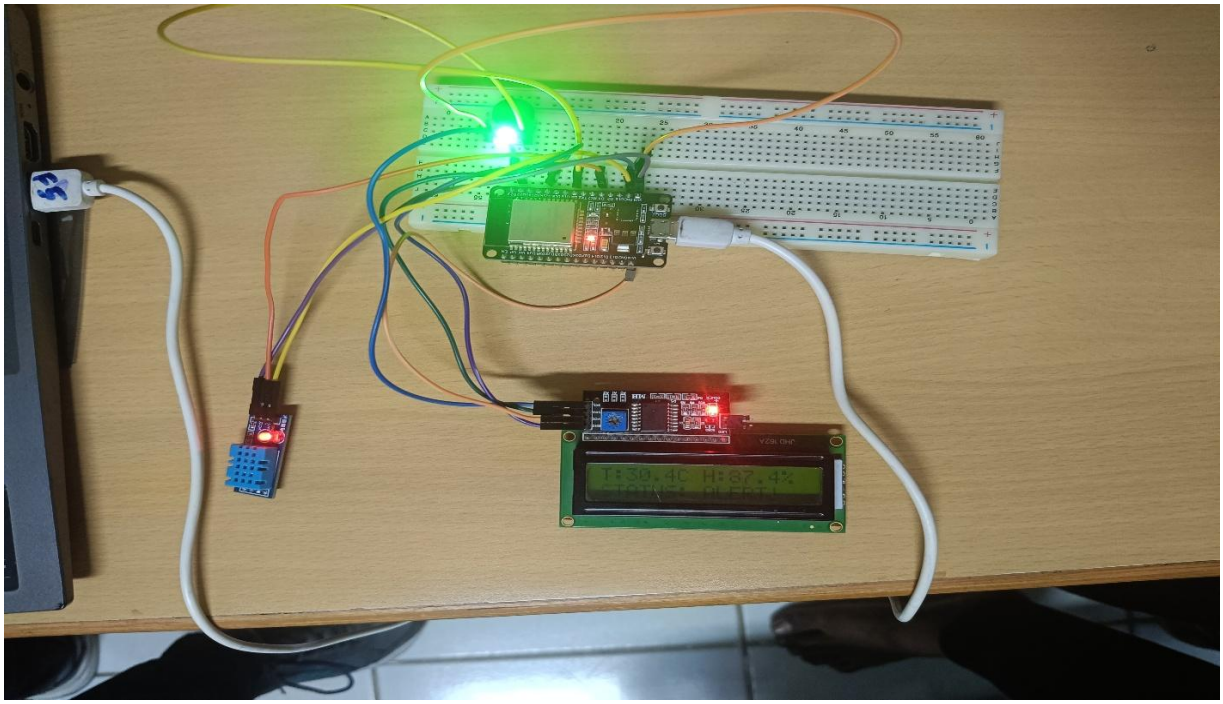

Fig. 5 Prototype — alternate/close-up view

## 12. Results & Performance Analysis

Once wiring and configuration issues were resolved, the system performed as designed:

- Sensor Responsiveness**:** The DHT11 delivered reliable temperature and relative humidity metrics updating every 2 seconds, displaying prompt shifts during temperature disturbancesThe LCD correctly displayed the live gas value on line 1 and the current status ("GOOD"/"BAD") on line 2, updating every second.

- Local Alert Performance: The buzzer and LCD alert toggled synchronously within $\leq 2$ seconds of threshold crossing.

- Local UI Verification: The $16 \times 2$ LCD clearly rendered dual metrics on row 1 and dynamic alert status on row 2 without flicker.

- Cloud Telemetry: Telemetry payloads were published at 20-second intervals over MQTT without broker dropped packets. ThingSpeak rendered continuous time-series curves fortemperature and humidity while the dashboard Lamp Widget correctly indicated status transistions.

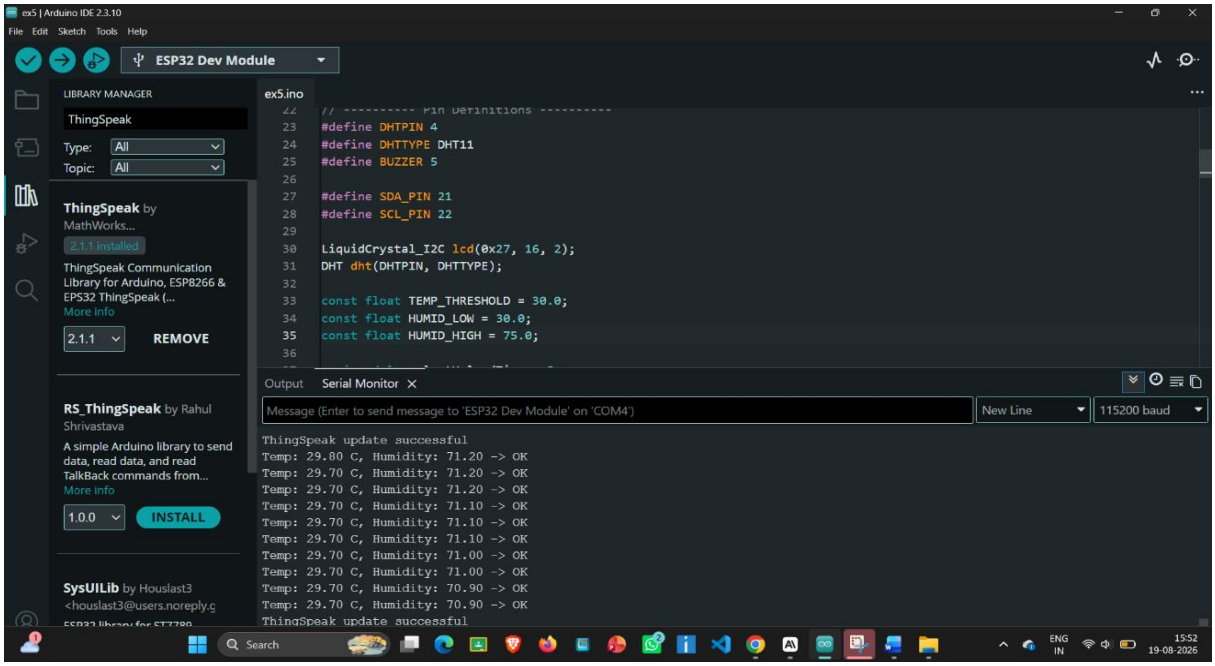


Fig. 6 Serial Monitor — live temperature & humidity readings and ThingSpeak upload log

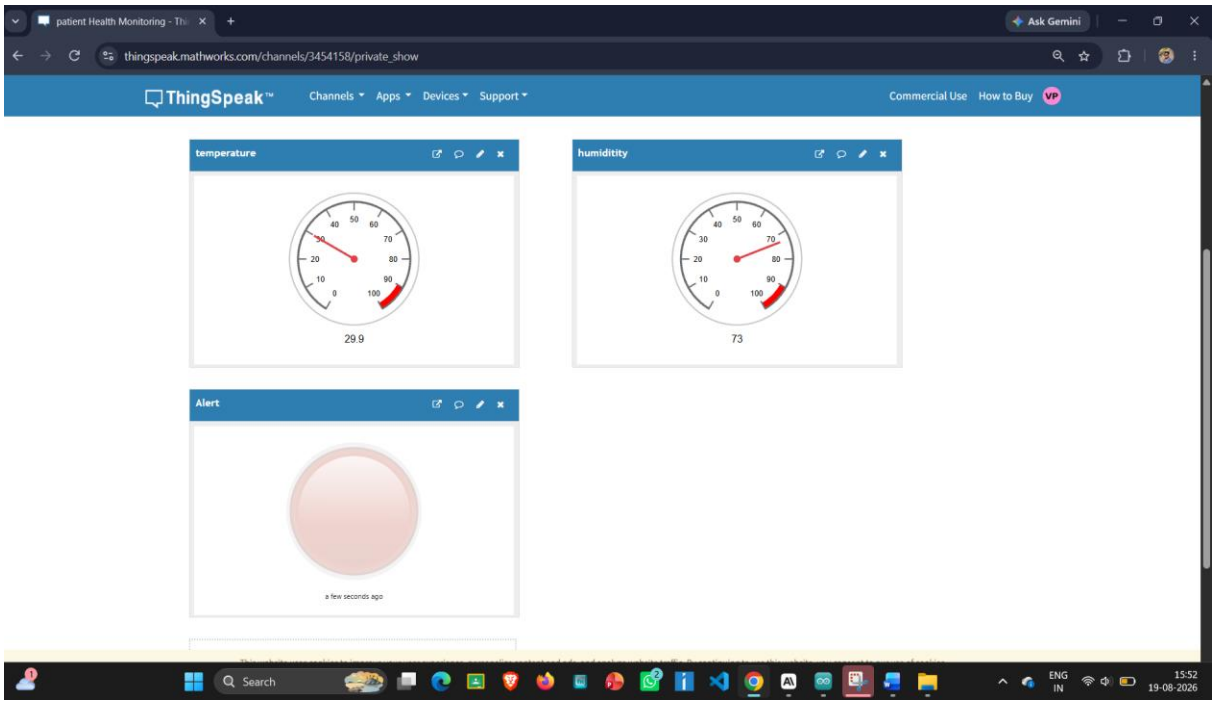


Fig. 7 ThingSpeak dashboard — live data with NORMAL status (lamp green/off)

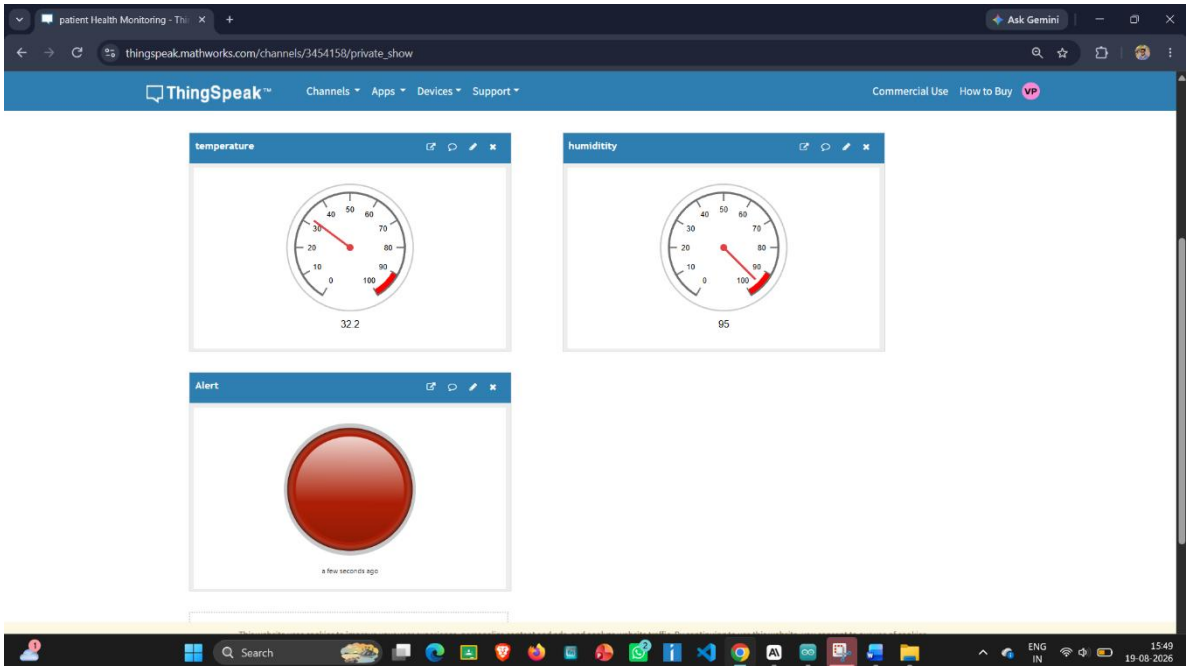


Fig. 8 ThingSpeak dashboard — live data with ALERT status (lamp red/on)